

Day 1 종합 실습

데이터 수집 미니 파이프라인 실행결과

관교캠 · 5반 · 홍은비

프로젝트 목표

Open-Meteo, Countries.dev, ip-api를 asyncio.gather()로 동시에 수집하고, Pydantic v2로 검증한 뒤 CSV·Parquet 저장 성능을 비교하는 실무형 파이프라인 구현

평가 항목	실행 결과
환경 구성	venv + requirements.txt 설치 완료
비동기 수집	API 3개 HTTP 200 / asyncio.gather() 사용
스키마 검증	날씨 72건·국가 1건·IP 1건 / 오류 0건
저장 비교	CSV·Parquet 각 3개 및 성능 요약 생성
품질 검사	pytest 3 passed / ruff 오류 없음

1. 실행 결과 화면

실제 API 수집, Pydantic 검증, pytest 및 ruff 실행 결과



```
$ python main.py
```

```
(API 비동기 수집)
```

- weather: 정상 응답 (HTTP 200)
- country: 정상 응답 (HTTP 200)
- ip_location: 정상 응답 (HTTP 200)

```
(Pydantic v2 검증)
```

- weather: 정상 72건
- country: 정상 1건
- ip_location: 정상 1건
- 검증 오류: 0건

```
Day 1 종합 실습 파이프라인을 완료했습니다.
```

```
$ pytest
```

```
3 passed
```

```
$ ruff check .
```

```
All checks passed!
```

세 요청은 동시에 시작되며, 가장 긴 개별 응답을 기다리는 동안 다른 요청도 함께 진행된다. 수집 결과는 API별 모델로 검증했으며 범위를 벗어난 값은 저장 전에 차단하도록 구현했다.

2. CSV·Parquet 성능 비교

데이터셋	형식	행	쓰기(초)	읽기(초)	크기
weather	CSV	72	0.000378	0.000507	4,843
weather	Parquet	72	0.000709	0.000716	4,677
country	CSV	1	0.000105	0.000191	130
country	Parquet	1	0.000256	0.000370	3,946
ip_location	CSV	1	0.000121	0.000225	190
ip_location	Parquet	1	0.000354	0.000435	6,095

결과 분석

날씨 데이터에서는 Parquet 파일 크기가 CSV보다 작았지만, 현재 데이터가 72행으로 작아 읽기·쓰기 시간 차이는 매우 작았다. 국가와 IP 데이터는 각각 1행이므로 Parquet 메타데이터 비용이 상대적으로 크게 나타났다. 대용량·다중 컬럼 데이터에서는 타입 보존과 압축 장점 때문에 Parquet의 효율이 더 뚜렷해질 것으로 판단한다.

코드 품질 및 개선 의견

수집·변환·검증·저장을 파일별로 분리해 책임을 명확히 했고, HTTP 오류·JSON 오류·검증 오류·저장 오류를 단계별로 처리했다. 향후에는 재시도와 지수 백오프, 날짜별 파티셔닝, 구조화 로그, 중복 방지 키를 추가해 운영 안정성을 높일 수 있다.

제출 체크리스트

✓ 전체 코드 및 requirements.txt ✓ API 3개 동시 수집 ✓ Pydantic v2 검증 ✓ CSV·Parquet 저장 ✓
pytest:ruff 통과 ✓ 실행결과 및 본인 의견